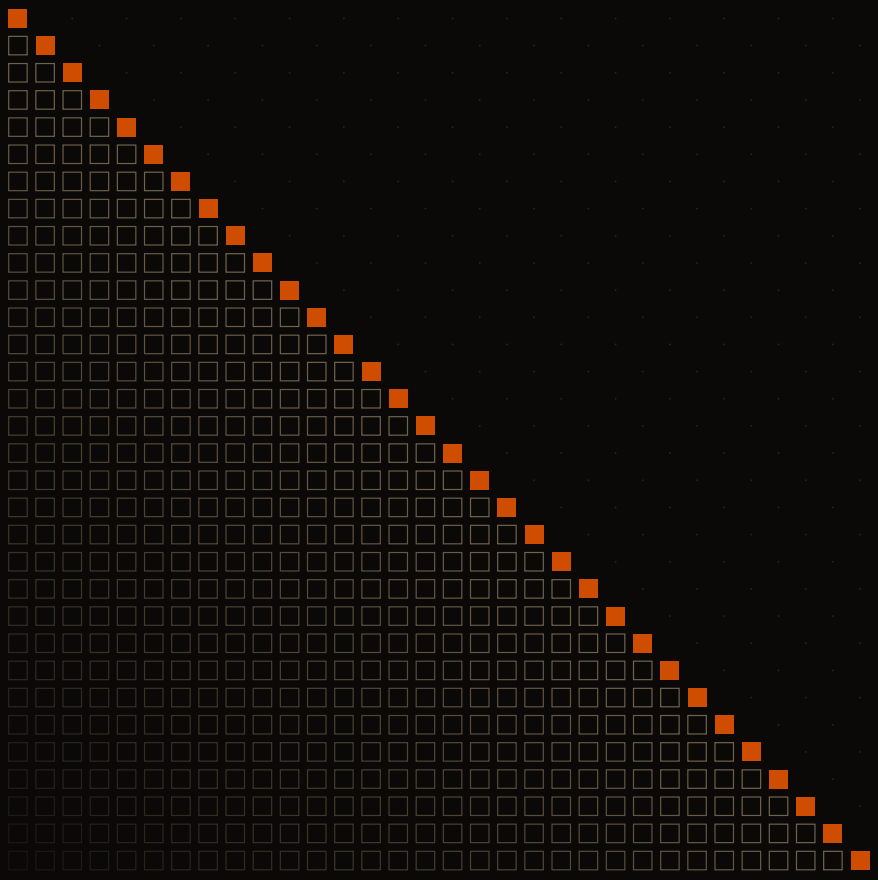


CAUSAL ATTENTION, N = 1024 · ONE SQUARE PER 32 × 32 TILE · NOTHING N × N WRITTEN DERIVED, NOT WRITTEN

Two Primitives Are Enough

Deriving flash attention, its backward pass, and whole-model kernels from affine views and monoid comprehensions

CAM PEDERSEN · MUNICIPAL ROBOTICS

Abstract

Deep learning frameworks reach peak speed through hand-written kernels: one per operator, per backend, per fusion pattern. Each kernel freezes decisions (what to fuse, where data lives, how threads share work) that depend on the model, the shape and the chip. We describe the tensor IR in tang, a Rust differentiable-computing library, which replaces those kernels with two primitives: views, affine maps from an iteration space to tensor coordinates, and comprehensions, reductions under a monoid whose body is a scalar expression graph. Fusion is one substitution rule. Flash attention is not written; it falls out of a rewrite that recognizes online softmax as a monoid, plus one staging rule. Gradients of comprehensions are comprehensions, so the backward pass, including a flash-style attention backward with no $n \times n$ intermediates, comes from the same passes. Number formats and thread mappings are schedule fields that cannot change the result, and a per-kernel autotuner chooses among them on the device. The same graphs lower to WGSL, CUDA (with `mma.sync` tensor cores), Metal (with simdgroup matrices), OpenCL, GLSL and C. On an RTX 3090 a transformer block's forward pass derived this way runs 1.37× faster than PyTorch with TF32, and a whole Qwen3-8B 4-bit decode step runs as one IR graph at 55% of memory-bandwidth speed of light. The compiler contains no operator-specific code.

1 Introduction

A modern model is a few dozen distinct operators. A modern framework is thousands of kernels, because every pair of (operator, backend) needs one, and every profitable fusion such as attention with its softmax, a matmul with its bias and activation, or a norm with the projection that follows needs another. FlashAttention[1] is the clearest case: a large speedup that came from rewriting one operator by hand so its $n \times n$ score matrix never reaches memory. Its backward pass took another hand-written kernel, and each new chip generation takes another rewrite.

Hand kernels are forks. Each re-derives the same arithmetic and freezes a schedule that was right for one shape on one device. We measured how often that schedule is wrong. Causal attention at $n = 512$ on an Apple M4 Max is fastest as the naive graph with n^2 threads, because the scores fit in cache and parallelism is the bottleneck. At $n = 1024$ it wants the tiled flash schedule. Matmul at $128 \times 576 \times 1536$ wants a 16×16 shared-memory tile; at 1024^2 the same tile loses. Rules of thumb tuned on the M4 picked the wrong matmul schedule on an NVIDIA RTX 3090. The winning schedule depends on the graph, the chip and the size, not on the operator.

This paper argues that a whole model should go through an IR small enough to reason about, and that everything a hand kernel encodes can be recovered as either an algebraic rewrite or a schedule choice. tang's tensor IR (`tang_compile::tensor`, about 24k lines of Rust) has two primitives:

- **Views**, affine index maps that express broadcast, transpose, slicing, striding, padding and per-head column selection without touching memory.
- **Comprehensions**, $\text{out}[i] = \oplus_k \text{body}(x_0[v_0(i,k)], x_1[v_1(i,k)], \ldots)$, where `body` is a scalar expression graph and $\oplus$ is a monoid.

Contributions:

1. An IR in which fusion, flash attention, logsumexp and Welford variance are instances of two general rewrites (§3).
2. Reverse-mode differentiation that maps comprehensions to comprehensions, including the online-softmax monoid and a proof procedure that removes the stabilizing max's gradient. Together with nested contractions, this yields an attention backward with no $n \times n$ tensors (§4).
3. Precision as a schedule, with an emulating interpreter that predicts a format choice's error before it runs (§5).
4. Lowerings to six targets from shared emitters, and measurements against hand kernels, PyTorch and the memory-bandwidth limit on three chips (§6, §7).

2 Background: tang

tang is a `no_std` Rust library that started as a geometry and physics math crate. One `Scalar` trait covers `f32`, `f64` and dual numbers, so a simulation written once yields exact derivatives by changing a type parameter. `tang-expr` traces Rust closures into a scalar DAG of nine RISC operations, which it can differentiate symbolically, simplify and compile. The tensor IR uses that scalar DAG as its body language, so every

comprehension body can be differentiated, simplified and compared structurally. Higher crates (`tang-gpu`, `tang-llm`, `tang-train`) run programs the IR produces.

3 The IR

3.1 Views

A view maps a point of an iteration space to a source tensor's coordinates with one affine form per source dimension, offset $+ \sum_j c_j \cdot \text{idx}_j$. Broadcast is a zero form, transpose a permutation, slice an offset, stride a coefficient. A form with two terms expresses "head h reads columns $64h + 2i$". Views compose by substitution. They are the only way a comprehension reads its inputs, so every layout decision is a view and every view can be analyzed.

A view may be guarded: an out-of-range read returns the monoid's identity (zero) rather than raising a shape error. Padded, dilated and transposed (polyphase) convolutions and overlap-add are guarded reads, with no padding operator. The gradient of a guarded read removes a coordinate rather than scattering. For $x[c, t + 3k - 9]$, $\partial x[c, \tau]$ reads $\partial y[o, \tau - 3k + 9]$ through a guard, so padding contributions never reach x.

Grouped-query attention's head / heads_per_group is not affine. The IR expresses heads as two dimensions, [groups, per_group], and the views do the rest. Dimensions can be fixed or symbolic (`Dim::Sym("seq")`). Construction, rewriting and differentiation work with symbols, and `bind` makes them concrete for scheduling and lowering.

3.2 Comprehensions and monoids

The only compute operation is

$$\text{out}[i_1, \ldots, i_p] = \bigoplus_{k_1, \ldots, k_r} \text{body}(x_0[v_0(i, k)], \ldots, x_m[v_m(i, k)])$$

A map is a comprehension with no reduce dimensions. Sum and max are monoids. Matmul is $\sum_k a[i, k] b[k, j]$. Attention scores, convolution and gather are the same operation with different views. The monoid's `combine` is itself a scalar graph, and a comprehension may carry tuple-valued state. Online softmax is the monoid on (m, l, acc); arg-max is the monoid on $(\text{index}, \text{value})$.

3.3 Rewrites

Fusion is one rule. A map is a pure function of its inputs, so `substitute_maps` splices its body into every consumer and composes the views. Map→map, map→reduce and transpose→anything are the same case. The rule is cost-aware: a map is not recomputed inside a reduction unless it is cheap. Reductions are never substituted. They are inlined by a schedule decision, which separates algebra from scheduling. A row-major reshape that only merges dimensions is a view, not a copy.

Epilogues ride the reduction. A map that reads a reduction point for point (a bias, an activation, the acc $/l$ that finishes softmax) runs on the thread that holds the final accumulator.

Siblings share a pass. Two reductions over the same space and monoid with the same readers become one reduction under the product monoid (`merge_siblings`). The gate and up projections of a SwiGLU MLP become one tuple-valued matmul, the activation becomes its epilogue, and the normalized input is read once.

Flash attention is a rewrite. `stabilize_softmax` finds $m = \max_j s_j$ and every $\sum_j g \cdot \exp(s_j - m)$ over the same canonical s, with masks and scales allowed, and replaces them with one reduction under the online-softmax monoid. The value column of $\sum_j p_j v_j$ becomes a point dimension of the fused comprehension. Nothing $n \times n$ is materialized. Logsumexp, Welford variance and top-k are the same rewrite with different monoids. Figure 1 shows the attention case.

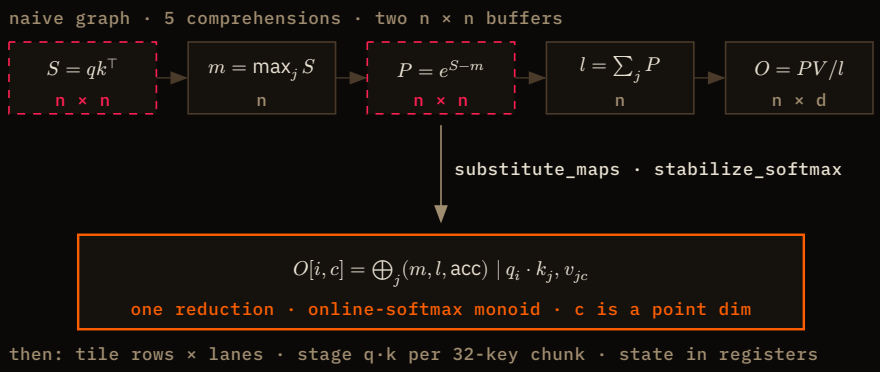


FIG 1 The rewrite that derives flash attention. The naive graph writes the scores and probabilities as $n \times n$ buffers (dashed). Fusion splices the maps into their readers; `stabilize_softmax` merges the max and both exponential sums into one reduction whose state is (m, l, acc). No attention-specific code is involved; scheduling then finds the tiled kernel.

3.4 Schedules

A comprehension says what to compute. Schedule fields say how, and they cannot change the answer:

- `inline`: recompute inside the consumer instead of writing a buffer.
- `split`: one workgroup per point, with threads striding the reduce space and tree-combining. This is valid for any monoid because combine is associative, including online softmax.
- `tile`, `tile_2d`: blocks of threads, each owning a register tile of points. The emitter tracks every address as an affine form over loop variables. Anything that does not depend on the lane axis is staged in workgroup memory once per reduce chunk. That covers buffer reads (K and V tiles, matmul operand tiles) and inlined sub-computations (the $q \cdot k$ scores).
- `precision`: storage and accumulation formats (§5).

`schedule_auto` encodes three rules distilled from measurement. `autotune_kernels` times each candidate mapping per materialized node on the device, using the graph's real inputs, and caches the winner by device and kernel source. With substitution and the online-softmax rewrite, the staging rule produces flash attention's real structure: rows × lanes per workgroup, cooperative score tiles, and online softmax in registers. Nobody wrote that kernel.

4 Gradients are comprehensions

For $y = \oplus_k \text{body}(x[v(i,k)])$, the gradient with respect to x through a pick view is found by renaming coordinates. The coordinates that v picks become x's output dimensions, the rest become reduce dimensions, and every other view is pushed through the same renaming. The body is differentiated by `tang-expr`'s symbolic `diff`. A dimension read through any other form (an offset, stride, diagonal or broadcast) stays an output dimension, and the body is gated by $[c = \text{form}]$ through `iota`, a node that turns coordinates into values. Max gates its term to the arg-max. The backward graph is an ordinary graph, so the same passes optimize it. Rotary embedding and GQA gradients come from the gated scatter with no special handling.

The online-softmax monoid has its own rule. With the forward's m and accumulators read back, $\partial \text{acc}_i / \partial s_j = g_i \exp(s_j - m) - \text{acc}_i [s_j = m]$, so the flash-fused graph differentiates and its gradients equal the naive graph's.

A stabilizing max has no gradient. Softmax is invariant to shifting its scores, so the $[s = m]$ terms sum to zero, but only in exact arithmetic. While those terms exist, the max must be materialized and masked tiles cannot be skipped. `backward` drops them when it can prove the loss does not depend on m. Each node downstream of m gets a degree in $t = \exp(-m)$, either $t^k h$ or $a + \kappa m$ with h, a free of m, using only identities such as $t^a t^b = t^{a+b}$ and $\sum_j t h_j = t \sum_j h_j$ when t's row does not depend on j. The output o/l then has degree 0. If every path from m ends at a degree-0 node, $\partial L / \partial m \equiv 0$ and the gradient is not built. The f64 evaluation of a training step is unchanged to 2.7×10^{-19} absolute at gradients of order 10^6.

Flash backward is nesting. `hoist_invariant_factors` distributes sums ($\sum_c \tilde{P} \cdot \text{dO} \cdot V \to \tilde{P} \cdot \sum_c \text{dO} \cdot V$). `schedule_nested_contractions` then inlines a short contraction (one reduce dimension of at most 128) into every tile that reads it. The attention backward reads $S = QK^\top$ and $\text{dP} = \text{dO}\ V^\top$ only through $\tilde{P}$ and dS, which the dQ, dK and dV tiles recompute per staged block on tensor cores. On CUDA this removed S, $\tilde{P}$, dP, the max over S and three correction kernels from a transformer training step, reducing 63 kernels to 49 and the summed best-per-kernel time from 1.80 to 1.22 ms at sequence length 512 (RTX 3090).

5 Precision is a schedule

f16 is a different way to run the same program, not a different program. Each node has a storage format, applied where a value is written to memory, and an accumulation format, applied to a reduction's running state after each combine. The graph stays exact math.

The interpreter predicts the error. `eval` is pure f64. `eval_emulated` rounds every stored value and every accumulator step to its scheduled format (round to nearest even, subnormals, overflow), with body arithmetic in f64. An `ErrorBudget` accepts or rejects a schedule before it runs. Emulation matches the GPU's f16 kernels to 1 ulp when the reduction order is the same. Reduction order matters: AdaIN's variance over $T = 2000$ with an f16 accumulator has error 2.8×10^{-4} under a 256-thread split and 1.3×10^{-2} with one thread per row, while the sequential emulation predicts

7.9×10^{-3}. Emulation models the worst case. `tune_precision` lowers buffers to half precision one move at a time and keeps a move only if the program stays within budget and gets faster.

6 Lowering

Shared emitters produce the point, split and tiled schedules, register tiles, guarded views, epilogue fusion and tuple monoids once. A dialect switch then handles buffer access and surface syntax.

- **WGSL** (portable default, through wgpu on Metal, Vulkan and DX12). f16 storage is packed two per `u32`, because the toolchain rejects `enable f16`.
- **CUDA.** Matmul-shaped tiles run on `mma.sync` (TF32 m16n8k8, or f16/bf16 m16n8k16 when the operands are half precision) with `cp.async` staging rings. Block shapes are tuning candidates. `cuda_flash_kernel` recognizes the online-softmax comprehension and keeps O, the row max and the partial sums in `mma` registers. The score accumulator is reused as the A operand of $O += PV$ after permuting keys within each 8-wide step, which is legal because the reduction is associative.
- **Metal (MSL).** Native `half` and `bfloat`. Contractions with f16 operands run on `simdgroup_half8x8` fragments. Reduce dimensions are flattened, so convolution becomes implicit GEMM, with dilated inputs staged once per chunk as a zero-padded, phase-de-interleaved window.
- **OpenCL and GLSL** for Adreno GPUs that have neither CUDA nor Metal. On one Adreno driver, WGSL is routed through GLSL because the direct SPIR-V path was miscompiled.
- **C**, for batch-1 control loops where a GPU round trip costs more than the arithmetic. A comprehension already is a loop nest, so `to_c` emits it directly with a vectorizable innermost point dimension, and `tang_run()` performs no allocation.

`Program::tuned` times each kernel's native lowering against the portable one and keeps the faster. Kernel sources are canonicalized, so the same kernel at another layer is the same string and compiles once.

7 Evaluation

Setup. Apple M4 Max (Metal), NVIDIA RTX 3090 (Vulkan and CUDA 12), and a Qualcomm QCS8550 in a Booster K1 humanoid robot (Adreno 740, Cortex-X3). PyTorch 2.10 + cu128 with CUDA events. Unless stated otherwise, tang times are kernel-only. Several runs shared the GPU with other tenants. We mark those, and report minima rather than means, because contention only adds time. Every derived result is checked against an f64 interpreter; agreement is 10^{-6} to 10^{-4} relative, depending on format.

7.1 Derived vs hand-written kernels

tang shipped hand-written WGSL kernels before the IR existed, which makes a controlled comparison possible: same runtime, same chip, same inputs. Table 1 is

one consistent run on the M4 Max, including about 1.8 ms of submit and readback per measurement.

TABLE 1 Hand-written vs derived on M4 Max (Metal via WGSL). The hand training step's attention backward ran on the CPU because no one had written a GPU version. The derived step had no such gap, because none of it was written.

OP	HAND	DERIVED	SCHEDULE
causal attention $n = 1024$	10.9 ms	**3.31**	naive (n^2 threads)
causal attention $n = 512$	4.91	**2.02**	auto (tiled)
matmul 1024^2	6.68	5.49	auto
rmsnorm 2048^2	2.52	2.70	split
transformer block, seq 512	23.9	**10.1**	per-kernel autotune
decode block, q 1 / kv 1024	8.80	**3.28**	per-kernel autotune
training step, seq 128	179	**7.96**	auto

Every hand kernel is matched or beaten. The schedules that win differ across operators and sizes. On the 3090 under Vulkan, the derived transformer block first lost to the hand path. Three chip-agnostic fixes followed: cost-aware substitution, keeping reductions schedulable under their epilogues, and a bank-conflict-free tile layout. After them, the autotuned block won there too (2.5 vs 3.1 ms at seq 128, 6.2 vs 10.4 ms at seq 512).

7.2 Against PyTorch

Table 2 tracks a 9-head GQA transformer block ($d = 576$, ff $= 1536$, rotary embeddings, SwiGLU) on the RTX 3090 as the backends matured. Through WGSL, which has no access to tensor cores, derived kernels stayed 2.5–4× behind PyTorch. With the CUDA lowering, flash attention on `mma.sync` and the backward passes of §4, the forward block passes PyTorch (Figure 2).

TABLE 2 Transformer block on the RTX 3090. The CUDA column and PyTorch were measured in interleaved rounds under `flock`, best of six; times in ms; PyTorch's 4-layer step uses fused AdamW. The WGSL columns come from earlier runs and are shown to make the trajectory visible.

BLOCK, SEQ	WGSL FIRST	WGSL FUSED	CUDA NOW	TORCH FP32/TF32
fwd 128	1.97	1.06	**0.178**	0.306 / 0.286
fwd 512	4.41	3.34	**0.328**	0.602 / 0.449
fwd+bwd 128	5.0	3.15	**0.552**	1.196 / —
fwd+bwd 512	10.1	8.19	**1.467**	1.719 / —
4 layers, step 128	—	—	**3.92**	4.40

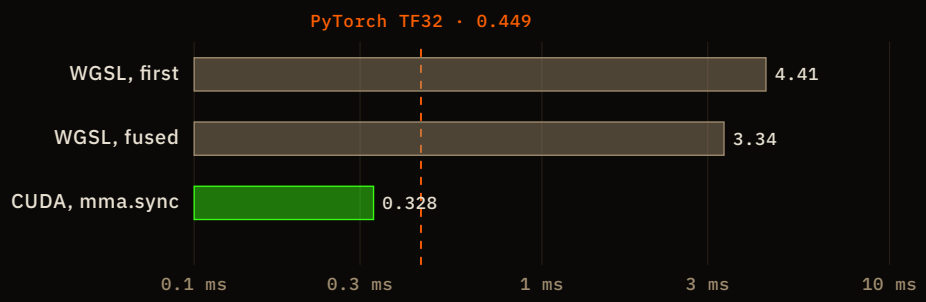


FIG 2 Transformer block forward, seq 512, RTX 3090, log scale. The same derived graph under three generations of lowering; only the last reaches tensor cores. Dashed: PyTorch with TF32.

On the CUDA path, the derived inference attention kernel is 0.032 ms at seq 512, down from 0.178 ms before the `mma.sync` rewrite. The seq-512 forward block is 1.37× faster than PyTorch with TF32. The training step is at parity or slightly ahead. We do not claim a general win over cuBLAS: a lone fp32 matmul at 1024^2 through WGSL reaches 7.6 TFLOPS against cuBLAS's roughly 19. The gain comes from fusion across operator boundaries that a library of separately tuned operators cannot do.

7.3 A whole model as one graph

Qwen3-8B with MLX 4-bit weights runs as one IR graph per decode step, lowered by `to_cuda_decode` into 506 kernels and captured as a CUDA graph. Nibble unpacking and the $q \cdot s + b$ dequantization are a map the GEMV reads. That map is inlined and evaluated in registers as weights stream in. Figure 3 shows where the remaining time goes. The KV-cache update is a `select` over a runtime position, which the lowering turns into loop bounds, so a step touches only live cache rows.

TABLE 3 Measured while another job held about 98% of SM time, so the true numbers are likely higher. Teacher-forced on 64 greedy tokens against an exactly dequantized f32 CPU reference: max logit error 1.7×10^{-4}, with 64/64 arg-max agreement.

QWEN3-8B, 3090	DECODE, SHORT CTX	DECODE, 4K CTX	PREFILL 512
speed of light (936 GB/s)	220 tok/s	192 tok/s	—
tang IR, device step	**122 tok/s** (55%)	**104 tok/s** (54%)	**774 tok/s**

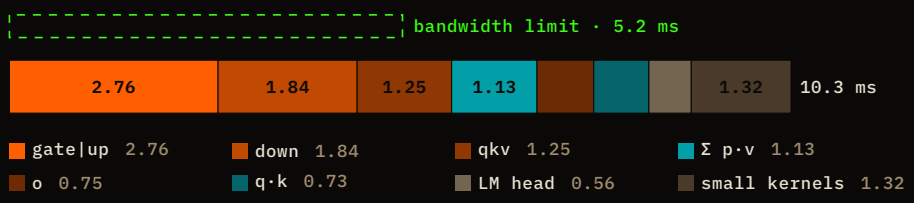


FIG 3 Where a Qwen3-8B decode step goes at 4k context: the sum of isolated kernel launches (10.3 ms; the captured graph runs in 9.6). The large GEMVs dominate; about 400 small reductions and maps add 1.3 ms of mostly fixed launch cost. Dashed: the time to stream the weights and KV cache once at 936 GB/s.

The same graph builder lowers to Metal. There, prefill and decode programs share one copy of the weights through buffer binding, prefill can start from a nonzero offset (300 + 200 tokens is bit-identical to 500), and the next token is chosen on the device by an arg-max monoid, so decoding reads back 8 bytes per token.

7.4 Edge hardware

On the K1's Adreno 740, a derived matmul reaches 322 GFLOP/s at 1024^3, which is 21% of the measured fp32 peak, and a full transformer forward pass plus all seven weight gradients runs in 19.1 ms. Rules tuned for desktop GPUs under-fill the Adreno's six compute units. The best schedule there (an 8×8 register tile with 8-deep chunks) is 1.45× better than `schedule_auto`'s choice, which again argues for search over remembered rules. Through `to_c` on one Cortex-X3 core, a batch-1 GRU policy step (6 contractions plus gate math, fused into one kernel) takes 22.5 µs median and a locomotion MLP takes 6.7 µs, both within 6×10^{-7} of f64.

8 Verification

Speed claims depend on every kernel being correct. `tang-verify` runs each GPU kernel against its CPU oracle on the same input, starting from the device's own state. Before each kernel runs, it fills the kernel's outputs with NaN, so unwritten memory cannot pass as a plausible 0. When a check fails, it identifies the responsible kernel by provenance: whose correct values ended up in this buffer. It was built after four structural bugs in a hand-written engine slipped past every unit test. One was a byte offset added where a float offset was meant, which wrote k's values over q. Derived kernels are less exposed to that class of bug, because addresses come from affine forms rather than pointer arithmetic. The IR path also has a structural oracle: the f64 interpreter runs the identical graph.

9 Related work

Halide[2] separated algorithm from schedule. TVM[3] and Ansor[4] added learned and searched schedules. Tensor Comprehensions[5] used polyhedral comprehensions. tang inherits the separation and the search. What it adds is monoid-valued comprehensions whose combine is an analyzable scalar graph, which turns online softmax into a rewrite rather than a template, and differentiation inside the IR, so the backward pass is also derived and scheduled. Triton[6] makes hand kernels easier to write; tang tries to make them unnecessary. FlashAttention[1,7] and its backward are hand derivations of what §3 and §4 produce mechanically. Online softmax itself is due to Milakov and Gimelshein[8]. XLA and TorchInductor fuse elementwise chains around library calls. tang has no library calls to fuse around.

10 Limitations

- **Measurement conditions.** Several key runs (the Qwen3 decode and the flash-backward kernels) were taken on shared GPUs. We report minima, which bound the code's speed from below, but they are not idle-machine numbers.
- **No real cost model.** `schedule_auto` is three measured rules, and the autotuner is the honest answer until a cost model exists.
- **Non-affine indexing.** Paged KV caches and MoE routing need a gather view that reads an index tensor, and that view blocks some fusion.

- **Scatter gradients** reduce over the whole source coordinate they gate on, so a stride-2 read pays 2× in the backward pass.
- **Body arithmetic** is f32 on every path. Storage and accumulators honor f16.
- **Distributed placement** exists as a rewrite (sharded reductions joined by an all-reduce expressed as a comprehension over the monoid's combine), but transport goes through the host.

11 Conclusion

Two primitives, affine views and monoid comprehensions, are enough to express a transformer, its gradient and its quantized inference. Fusion, flash attention and the flash backward are rewrites and schedule choices over those primitives, not kernels. Because the schedule is searched per chip, the same source beats hand kernels on Metal, Vulkan and an embedded Adreno, and passes PyTorch on a transformer block on CUDA. A hand-written kernel does not disappear in this design. It becomes an intrinsic: a known-fast implementation of one comprehension shape on one chip, which the compiler uses when it wins and replaces when it doesn't.

> ASIDE · REPRODUCIBILITY
> Every number in this paper comes from `BENCHMARKS.md` in the tang repository (snapshot in `data/`), which records the command, date and machine state for each measurement. The IR is in `crates/tang-compile/src/tensor*.rs`; its design notes are in `docs/tensor-ir.md`.

References

1. Dao, T., Fu, D. Y., Ermon, S., Rudra, A. & Ré, C. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. in Advances in Neural Information Processing Systems (2022).
2. Ragan-Kelley, J. et al. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines. in PLDI (2013).
3. Chen, T., Moreau, T., Jiang, Z. & others. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. in OSDI (2018).
4. Zheng, L., Jia, C., Sun, M. & others. Ansor: Generating High-Performance Tensor Programs for Deep Learning. in OSDI (2020).
5. Vasilache, N., Zinenko, O., Theodoridis, T. & others. Tensor Comprehensions: Framework-Agnostic High-Performance Machine Learning Abstractions. arXiv preprint arXiv:1802.04730 (2018).
6. Tillet, P., Kung, H. T. & Cox, D. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. in MAPL (2019).
7. Dao, T. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv preprint arXiv:2307.08691 (2023).
8. Milakov, M. & Gimelshein, N. Online normalizer calculation for softmax. arXiv preprint arXiv:1805.02867 (2018).